Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Compute Services

Study Guide

AMARJIT SEAL
AI&DS, PIET
Parul University

UNIT 1: Infrastructure as a Service (IaaS)

1.1 What is IaaS?

Infrastructure as a Service (IaaS) is a cloud computing service model that provides virtualized computing resources over the internet. In IaaS, the cloud provider manages and maintains the physical hardware (servers, storage, networking), while the consumer manages the operating system, middleware, applications, and data.

Key Characteristics of IaaS

- Resources available as a service — compute, storage, network on-demand
- Highly scalable — scale up or down as per workload
- Dynamic and flexible — provision/de-provision in minutes
- Multi-tenant — multiple users share underlying hardware
- Pay-as-you-go pricing — billed for resources consumed
- Self-service model — users manage via a web portal or API

IaaS vs PaaS vs SaaS — Quick Comparison

Aspect	IaaS	PaaS	SaaS
User manages	OS, Middleware, App, Data	App, Data	Only uses app
Provider manages	Hardware, Network, Virtualization	Everything below App	Everything
AWS Example	Amazon EC2	Elastic Beanstalk	Gmail / Office 365
Flexibility	Highest	Medium	Lowest
Control	Full	Partial	None

Advantages of IaaS

- No capital expenditure — reduces hardware investment
- Flexibility to scale infrastructure on demand
- Rapid deployment of resources
- Disaster recovery and business continuity options
- Geographic distribution — regions and availability zones
- Access to advanced hardware without ownership costs

Disadvantages of IaaS

- Security and compliance management remains with user
- Requires skilled IT staff to manage OS and middleware
- Vendor lock-in risks
- Network dependency — performance tied to internet

IaaS Providers

- Amazon Web Services (AWS) — Amazon EC2, S3, VPC
- Microsoft Azure — Azure Virtual Machines
- Google Cloud Platform — Google Compute Engine
- IBM Cloud — IBM Virtual Servers
- DigitalOcean — Droplets

UNIT 2: Virtual Machines in the Cloud

2.1 Virtualization Concept

Virtualization is the process of creating a software-based (virtual) representation of hardware resources such as servers, storage, and networks. It allows multiple operating systems and applications to run concurrently on a single physical machine.

Types of Virtualization

- Full Virtualization — complete simulation of hardware (e.g., VMware)
- Para-Virtualization — guest OS is modified to be aware of hypervisor (e.g., Xen)
- OS-Level Virtualization — containers share host OS kernel (e.g., Docker)
- Hardware-Assisted Virtualization — CPU features support VM isolation (Intel VT-x, AMD-V)

Hypervisor Types

Type 1 (Bare Metal): Runs directly on hardware. Examples: VMware ESXi, Microsoft Hyper-V, Xen.

Type 2 (Hosted): Runs on top of a host OS. Examples: VirtualBox, VMware Workstation.

2.2 Virtual Machines in AWS Cloud

Amazon EC2 is the primary IaaS service for virtual machines in AWS. Each EC2 instance is a virtual machine running on AWS physical servers using Xen or Nitro hypervisors.

Key Components of a Cloud VM

- Instance — the virtual machine itself
- AMI (Amazon Machine Image) — template with OS + pre-installed software
- Instance Type — defines vCPU, RAM, storage, and network capacity
- Security Group — virtual firewall controlling inbound/outbound traffic
- Key Pair — SSH key credentials for secure instance access
- EBS Volume — Elastic Block Store for persistent storage
- Elastic IP — static IPv4 address assignable to an instance
- VPC — Virtual Private Cloud for network isolation

UNIT 3: Amazon EC2 (Elastic Compute Cloud)

3.1 Overview

Amazon Elastic Compute Cloud (Amazon EC2) is a web service that provides resizable compute capacity in the cloud. It is designed to make web-scale cloud computing easier for developers. EC2 eliminates the need to invest in hardware upfront so you can develop and deploy applications faster.

📌 **Note:** EC2 is an IaaS offering — the user manages the OS and applications, while AWS manages the physical hardware and virtualization layer.

3.2 Key Features of Amazon EC2

- Elastic — scale up/down compute capacity in minutes
- Flexible instance types — optimized for compute, memory, storage, or GPU
- Multiple pricing options — On-Demand, Reserved, Spot, Dedicated
- Integration with AWS ecosystem — S3, RDS, VPC, IAM, CloudWatch
- Availability Zones — deploy across multiple AZs for fault tolerance
- Amazon Machine Images (AMIs) — pre-built OS images for rapid launch

3.3 EC2 Instance Types

Family	Example Types	Use Case
General Purpose	t3, t4g, m5, m6g	Balanced compute, memory, networking — web servers, small databases
Compute Optimized	c5, c6g	High-performance computing, batch processing, gaming servers
Memory Optimized	r5, r6g, x1e	In-memory databases, real-time analytics (SAP HANA)
Storage Optimized	i3, d3	NoSQL databases, data warehousing, distributed file systems
Accelerated Computing	p3, g4, inf1	Machine learning, GPU rendering, HPC

3.4 EC2 Pricing Models

On-Demand: Pay for compute capacity per second/hour with no long-term commitments. Best for short-term, unpredictable workloads.

Reserved Instances: Significant discount (up to 75%) in exchange for 1 or 3-year commitment. Best for steady-state workloads.

Spot Instances: Bid on unused EC2 capacity. Up to 90% cheaper but can be interrupted. Best for fault-tolerant batch jobs.

Dedicated Hosts: Physical server dedicated to your use. Helps meet compliance requirements. Most expensive option.

Savings Plans: Flexible pricing model offering savings in exchange for consistent usage commitment.

3.5 Launching an Amazon EC2 Instance — Step-by-Step

1. Sign in to AWS Management Console and navigate to EC2 Dashboard.
2. Click 'Launch Instance'.
3. Choose an Amazon Machine Image (AMI) — e.g., Amazon Linux 2, Ubuntu, Windows Server.
4. Choose an Instance Type (e.g., t2.micro for Free Tier eligible).
5. Configure Instance Details — number of instances, VPC, subnet, IAM role.
6. Add Storage — specify EBS root volume size (default: 8 GB).
7. Add Tags — key-value pairs to identify and organize resources.
8. Configure Security Group — define inbound/outbound rules (e.g., allow SSH port 22, HTTP port 80).
9. Review and Launch — select or create a Key Pair for SSH access.
10. Click 'Launch' — instance will be in 'running' state within minutes.

📌 **Note:** For SSH access: chmod 400 keypair.pem && ssh -i keypair.pem ec2-user@<public-ip>

3.6 EC2 Storage Options

EBS (Elastic Block Store): Persistent block storage volumes. Data persists independently of the instance lifecycle. Supports snapshots.

Instance Store: Temporary (ephemeral) block storage physically attached to the host computer. Data is lost when the instance stops.

EFS (Elastic File System): Managed NFS file system, shared across multiple EC2 instances.

S3: Object storage — not directly attached; accessed via APIs.

3.7 Security Groups and Key Pairs

Security Groups act as a stateful virtual firewall at the instance level. They control inbound and outbound traffic. Key features:

- Inbound rules — control incoming traffic (protocol, port, source IP)
- Outbound rules — control outgoing traffic (default: allow all)
- Stateful — if inbound traffic is allowed, the response is automatically allowed
- Can be modified without restarting the instance

UNIT 4: Elastic Load Balancing (ELB)

4.1 What is Elastic Load Balancing?

Elastic Load Balancing (ELB) automatically distributes incoming application traffic across multiple Amazon EC2 instances, containers, IP addresses, and Lambda functions. It helps ensure that no single resource is overwhelmed, improving availability and fault tolerance.

4.2 Types of Load Balancers

1. Application Load Balancer (ALB): Operates at Layer 7 (HTTP/HTTPS). Routes traffic based on URL path, host headers, query strings. Best for web applications and microservices.

2. Network Load Balancer (NLB): Operates at Layer 4 (TCP/UDP). Handles millions of requests per second with ultra-low latency. Best for real-time applications.

3. Gateway Load Balancer (GWLB): Operates at Layer 3 (IP). Used to deploy, scale, and manage third-party virtual network appliances.

4. Classic Load Balancer (CLB): Legacy load balancer supporting both Layer 4 and 7. Being phased out in favor of ALB/NLB.

4.3 Key Features of ELB

- High Availability — distributes traffic across multiple Availability Zones
- Health Checks — automatically routes traffic only to healthy instances
- SSL/TLS Termination — decrypts HTTPS traffic, reducing load on instances
- Sticky Sessions — route same user to the same backend instance
- Integration — works seamlessly with Auto Scaling, EC2, ECS, Lambda
- Access Logs — detailed logs of requests for analysis
- CloudWatch Metrics — monitor request counts, latency, errors

4.4 How ELB Works

11. Client sends a request to the load balancer's DNS name.
12. ELB receives the request and checks which backend instances are healthy.
13. ELB forwards the request to a healthy EC2 instance using the routing algorithm.
14. The EC2 instance processes the request and sends the response back through ELB.
15. ELB forwards the response to the client.

📌 **Note:** ELB DNS name example: my-load-balancer-1234567890.us-east-1.elb.amazonaws.com

4.5 Health Checks

ELB performs periodic health checks on registered targets. If an instance fails a health check, ELB stops routing traffic to that instance and only resumes when the instance passes the check again.

- Protocol: HTTP, HTTPS, TCP, SSL

- Healthy threshold — number of consecutive successful checks to mark as healthy
- Unhealthy threshold — number of consecutive failed checks to mark as unhealthy
- Timeout — time to wait for a health check response
- Interval — time between health checks

UNIT 5: Auto Scaling

5.1 What is Auto Scaling?

AWS Auto Scaling monitors your applications and automatically adjusts capacity to maintain steady, predictable performance at the lowest possible cost. It ensures you have the right number of EC2 instances to handle the load at any given time.

5.2 Components of Auto Scaling

Launch Template / Launch Configuration: Specifies the AMI ID, instance type, key pair, security groups, and other parameters for launching new instances.

Auto Scaling Group (ASG): A collection of EC2 instances treated as a logical grouping for scaling purposes. Defines minimum, maximum, and desired capacity.

Scaling Policies: Define when and how to scale. Can be based on CloudWatch metrics, schedules, or predictive algorithms.

5.3 Scaling Types

Manual Scaling: User manually changes the desired capacity of the Auto Scaling Group.

Scheduled Scaling: Scale based on predictable load changes (e.g., increase instances every Monday morning).

Dynamic Scaling: Respond to changing conditions automatically. Includes Target Tracking, Step Scaling, and Simple Scaling policies.

Predictive Scaling: Uses machine learning to predict future traffic and proactively adjusts capacity.

5.4 Scaling Policies

Target Tracking Scaling: Maintain a specific metric value (e.g., average CPU utilization = 50%). ASG automatically adjusts to keep metric at target.

Step Scaling: Scale in steps based on metric thresholds. E.g., add 2 instances when CPU > 70%, add 4 instances when CPU > 90%.

Simple Scaling: Scale by a fixed amount when a CloudWatch alarm is triggered. Has a cooldown period after scaling.

5.5 Key Concepts

Minimum Capacity: The lowest number of instances that must be running at all times.

Maximum Capacity: The upper limit on the number of instances.

Desired Capacity: The ideal number of instances. ASG maintains this count, launching/terminating to meet it.

Cooldown Period: Time after a scaling activity where ASG does not start another scaling activity (default: 300 seconds).

Health Check Grace Period: Time to wait before starting health checks on newly launched instances.

5.6 Benefits of Auto Scaling

- Improved fault tolerance — replace unhealthy instances automatically
- Better availability — maintain required instance count across AZs
- Cost optimization — terminate unnecessary instances during low traffic
- Works with ELB — newly launched instances are automatically registered
- CloudWatch integration — scale based on any CloudWatch metric

UNIT 6: Serverless Computing & AWS Lambda

6.1 What is Serverless Computing?

Serverless computing is a cloud execution model where the cloud provider dynamically manages the allocation and provisioning of servers. The term 'serverless' is somewhat misleading — servers do exist, but developers don't need to manage them. You write code, deploy it, and the cloud handles the rest.

Serverless vs Traditional vs IaaS

Feature	Traditional Server	IaaS (EC2)	Serverless (Lambda)
Server management	User manages fully	User manages OS+App	None — AWS manages all
Scaling	Manual	Manual / Auto Scaling	Automatic, instant
Billing	Fixed (24/7)	Per hour/second	Per invocation + ms
Idle cost	High	Yes	Zero
Deployment	Complex	Moderate	Simple

6.2 AWS Lambda

AWS Lambda is a serverless, event-driven compute service that lets you run code without provisioning or managing servers. You pay only for the compute time you consume — there is no charge when your code is not running.

6.3 Characteristics of AWS Lambda

- Event-Driven Execution — Lambda functions are triggered by events (S3 upload, API call, DynamoDB stream, SNS message, etc.)
- No Server Management — AWS handles all server provisioning, maintenance, patching, and scaling
- Automatic Scaling — scales automatically from a few requests per day to thousands per second
- Pay-Per-Use Billing — charged based on number of requests and duration of execution
- Stateless Execution — each function invocation is independent; no state is preserved between calls
- Short Execution Duration — maximum execution time is 15 minutes per invocation
- Supported Runtimes — Node.js, Python, Java, Go, Ruby, .NET, and custom runtimes
- Memory Configuration — 128 MB to 10,240 MB; CPU is allocated proportionally
- Concurrent Executions — runs multiple instances in parallel to handle load
- VPC Integration — Lambda functions can access resources inside a VPC
- Layers — reusable code/libraries shared across multiple functions
- Environment Variables — configuration values passed securely at runtime
- Destinations — route function output to SQS, SNS, EventBridge, or another Lambda
- IAM Integration — fine-grained permissions via execution roles

6.4 How AWS Lambda Works

16. You write and upload your function code (as a ZIP file or container image).
17. You configure an event source (trigger) — e.g., an S3 bucket, API Gateway endpoint.
18. When the trigger fires, Lambda automatically allocates resources and runs your function.
19. Lambda manages execution environment, handles scaling, and collects metrics/logs.
20. You are billed for the number of requests and duration of execution (per millisecond).

6.5 Lambda Pricing

Requests: First 1 million requests/month are free; $0.20 per 1 million requests thereafter.

Duration: Charged based on GB-seconds (memory allocated × execution time). First 400,000 GB-seconds/month are free.

6.6 Lambda Use Cases

- Data processing — real-time file processing when uploaded to S3
- Web backends — REST APIs via API Gateway + Lambda
- Scheduled tasks — replace cron jobs using EventBridge Scheduler
- Stream processing — process Kinesis or DynamoDB Streams
- IoT backends — process messages from IoT devices
- Chatbots and Alexa skills
- Email processing via SES triggers

6.7 Creating and Configuring a Lambda Function

Step-by-Step via AWS Console

21. Open AWS Lambda Console → Click 'Create Function'.
22. Choose Author from Scratch.
23. Enter Function Name (e.g., 'MyFirstFunction').
24. Select Runtime (e.g., Python 3.12, Node.js 20.x).
25. Choose or create an Execution Role with necessary IAM permissions.
26. Click 'Create Function'.
27. In the 'Code Source' section, write or paste your function code.
28. Configure the Handler — entry point for your code (e.g., lambda_function.lambda_handler).
29. Set Memory (128 MB–10,240 MB) and Timeout (1 sec–15 min).
30. Add Environment Variables if needed.
31. Configure a Trigger (event source) — click 'Add Trigger' and select from S3, API Gateway, etc.
32. Test the function using the 'Test' tab with a sample event JSON.
33. Monitor execution via CloudWatch Logs and Lambda metrics.

📌 **Note:** Lambda function code example (Python): def lambda_handler(event, context): return {'statusCode': 200, 'body': 'Hello from Lambda!'}

Lambda Configuration Parameters

- Handler: The method in your code that processes events (e.g., index.handler)
- Memory: Affects CPU and network performance (128 MB to 10,240 MB)
- Timeout: Maximum execution duration (default 3 sec, max 15 min)
- Concurrency: Reserved or provisioned concurrency for performance control
- Dead Letter Queue (DLQ): SQS queue for failed async invocations
- VPC Config: Subnet IDs and Security Group IDs for VPC access
- Layers: Add shared libraries and dependencies

UNIT 7: PaaS — AWS Elastic Beanstalk

7.1 What is AWS Elastic Beanstalk?

AWS Elastic Beanstalk is a Platform as a Service (PaaS) offering that allows developers to quickly deploy and manage applications in the AWS Cloud without worrying about the infrastructure. You simply upload your application code, and Elastic Beanstalk automatically handles the deployment details including capacity provisioning, load balancing, auto-scaling, and application health monitoring.

📌 **Note:** Elastic Beanstalk = PaaS. You manage only the application and data; AWS manages everything below (OS, runtime, web server, network, scaling, monitoring).

7.2 Supported Platforms

- Java (Tomcat)
- Python (with Gunicorn or uWSGI)
- Node.js
- .NET (IIS on Windows)
- PHP
- Ruby
- Go
- Docker — single or multi-container
- Custom platforms using Packer

7.3 Key Features of Elastic Beanstalk

- Easy Deployment — upload a ZIP file or WAR; Beanstalk handles the rest
- Auto Scaling — automatically scales based on application demand
- Load Balancing — integrated with ELB for traffic distribution
- Health Monitoring — built-in dashboard and CloudWatch integration
- Managed Updates — applies platform patches and updates automatically
- Environment Tiers — Web Server Tier and Worker Tier
- Configuration Files — use .ebextensions for custom configuration
- No Additional Cost — you pay only for the AWS resources (EC2, RDS, ELB) used, not for Beanstalk itself
- Version Management — maintain and deploy multiple application versions
- Multiple Environments — run development, staging, and production environments independently

7.4 Elastic Beanstalk Architecture

Application: The top-level Elastic Beanstalk component. Contains environments, versions, and configurations.

Application Version: A specific labeled iteration of your deployable code stored in S3.

Environment: A collection of AWS resources running an application version. Two types: Web Server and Worker.

Environment Tier:

- Web Server Tier — handles HTTP requests. Uses EC2, ELB, Auto Scaling.
- Worker Tier — processes background jobs from an SQS queue.

Platform: The combination of OS, runtime, web server, and Beanstalk components.

7.5 Difference: Elastic Beanstalk vs EC2

Feature	EC2 (IaaS)	Elastic Beanstalk (PaaS)
Control Level	Full control over OS, middleware	Less control, more automation
Setup Complexity	High — configure everything manually	Low — upload code and go
Scaling	Manual or Auto Scaling setup required	Built-in auto-scaling
Monitoring	Set up CloudWatch manually	Built-in health dashboard
Best For	Full customization needed	Fast web app deployment
Cost	Pay for EC2 resources	Pay for underlying resources only

7.6 Deploying a Sample Application to Elastic Beanstalk

Method 1: AWS Management Console

34. Open AWS Elastic Beanstalk Console.
35. Click 'Create Application'.
36. Enter an Application Name (e.g., 'MyWebApp').
37. Select Platform (e.g., Python, Node.js, Java).
38. Choose 'Upload your code' and upload a ZIP file, OR select 'Sample application' to use AWS's demo.
39. Click 'Create Application' — Beanstalk provisions all resources automatically.
40. Wait 3–5 minutes. When complete, click the generated URL to access your app.

Method 2: AWS CLI (Elastic Beanstalk CLI — EB CLI)

- Install EB CLI: pip install awsebcli
- Initialize: eb init -p python-3.11 my-app --region us-east-1
- Create environment: eb create my-env
- Deploy: eb deploy
- Open app in browser: eb open
- Check status: eb status
- View logs: eb logs
- Terminate: eb terminate my-env

7.7 .ebextensions Configuration

The .ebextensions directory in your application source bundle lets you customize and configure the AWS resources in your environment. Configuration files use YAML or JSON format with .config extension.

- Install packages on EC2 instances
- Run shell commands during deployment
- Configure environment properties
- Create AWS resources (RDS, SQS, etc.) as part of the environment

UNIT 8: IMPORTANT EXAM QUESTIONS

Compiled from university question papers (AKTU, KTU, VTU, BATU, Mumbai University, Anna University) and common exam patterns.

Section A: Short Answer Questions (2–5 Marks)

Q1. Define Infrastructure as a Service (IaaS). Give two examples of IaaS providers. **[2 marks]**

Q2. What is Amazon EC2? List any four key features of EC2. **[3 marks]**

Q3. Differentiate between IaaS, PaaS, and SaaS with examples. **[5 marks]**

Q4. What is Elastic Load Balancing? List the types of load balancers in AWS. **[3 marks]**

Q5. What is AWS Auto Scaling? Name its key components. **[3 marks]**

Q6. Define serverless computing. How does it differ from traditional computing? **[3 marks]**

Q7. What is AWS Lambda? List any four characteristics of Lambda. **[4 marks]**

Q8. What is AWS Elastic Beanstalk? Which service model does it represent? **[2 marks]**

Q9. What is an Amazon Machine Image (AMI)? **[2 marks]**

Q10. Define Auto Scaling Group. What are minimum, maximum, and desired capacity? **[3 marks]**

Q11. What is the difference between an Application Load Balancer and a Network Load Balancer? **[3 marks]**

Q12. List the supported programming languages/runtimes in AWS Lambda. **[2 marks]**

Q13. What is a Security Group in EC2? How does it work? **[3 marks]**

Q14. What is the maximum execution timeout for an AWS Lambda function? **[2 marks]**

Q15. Define Target Tracking Scaling Policy. Give an example. **[3 marks]**

Section B: Medium Answer Questions (6–10 Marks)

Q16. Explain the concept of IaaS. What are its advantages and disadvantages? Compare IaaS with PaaS with suitable examples. **[8 marks]**

Q17. Explain Amazon EC2 in detail. What are the different EC2 instance types? Describe EC2 pricing models. **[10 marks]**

Q18. Explain the process of launching an Amazon EC2 instance step by step. What are the key configuration options? **[8 marks]**

Q19. Describe Elastic Load Balancing. Explain the different types of load balancers and their use cases. **[8 marks]**

Q20. Explain AWS Auto Scaling. Describe the different types of scaling policies with examples. **[10 marks]**

Q21. What is serverless computing? Explain the characteristics of AWS Lambda with examples of use cases. **[8 marks]**

Q22. Describe the steps to create and configure an AWS Lambda function. Explain Lambda pricing. **[8 marks]**

Q23. Explain AWS Elastic Beanstalk as a PaaS offering. Compare Elastic Beanstalk with Amazon EC2. **[8 marks]**

Q24. Explain the process of deploying a sample application to AWS Elastic Beanstalk step by step. **[6 marks]**

Q25. Explain EC2 storage options: EBS, Instance Store, EFS. How do they differ? **[6 marks]**

Section C: Long Answer / Essay Questions (15–20 Marks)

Q26. Explain in detail the concept of Infrastructure as a Service (IaaS). With a focus on Amazon EC2, describe virtual machines in the cloud, instance types, pricing models, storage options, and security configurations. Compare IaaS with PaaS and SaaS using AWS services as examples. **[16 marks]**

Q27. Explain Elastic Load Balancing and Auto Scaling in AWS. Why are these two services typically used together? Explain the different types of ELB and scaling policies. Describe a real-world scenario where both would be used. **[16 marks]**

Q28. What is serverless computing? Explain AWS Lambda in detail, covering its architecture, characteristics, event sources, programming model, and pricing. Include a discussion of how Lambda compares to EC2 for web application deployment. **[20 marks]**

Q29. Explain AWS Elastic Beanstalk as a Platform as a Service offering. Describe its architecture, supported platforms, deployment process, and configuration options. Explain the difference between Web Server Tier and Worker Tier. How does Elastic Beanstalk simplify application deployment compared to managing EC2 instances directly? **[20 marks]**

Q30. Write a detailed note on all AWS compute services covered in this unit: IaaS (EC2), Serverless (Lambda), and PaaS (Elastic Beanstalk). Compare all three in terms of control, scalability, pricing, management overhead, and best use cases. Provide a diagram where appropriate. **[20 marks]**

Section D: Previous University Exam Questions (Actual)

AKTU (Dr. APJ Abdul Kalam Technical University)

Q31. Explain Amazon EC2 and its basic features and related services. [KCS713 / 2022-23] **[10 marks]**

Q32. What is the use of CloudWatch in Amazon EC2? [KCS713 / 2022-23] **[2 marks]**

Q33. What are the service models available in cloud computing? [AKTU Sem VII] **[2 marks]**

Q34. Describe in detail about major Deployment Models and services for cloud computing. **[10 marks]**

KTU (APJ Abdul Kalam Technological University, Kerala — CST423)

Q35. What are the key benefits of Infrastructure as a Service (IaaS)? Write any two IaaS providers in the current market. [May 2024] **[3 marks]**

Q36. Explain Platform as a Service (PaaS) in terms of 'ready to use' environment. [CST423 / 2024] **[14 marks]**

Q37. Differentiate between Amazon S3 and Amazon EBS. [CST423 / 2024] **[3 marks]**

BATU (Dr. Babasaheb Ambedkar Technological University, Lonere)

Q38. What cloud providers do for IaaS? Explain scope of responsibilities between provider and consumer. [Winter 2023] **[6 marks]**

Q39. Explain Amazon EC2 and its basic features and related services. [B.Tech Supplementary 2023] **[6 marks]**

General University Pattern Questions

Q40. Write a short note on Elastic Load Balancing (ELB) and Auto Scaling. **[5 marks]**

Q41. Explain the concept of Pay-as-you-go pricing in the context of AWS Lambda. **[4 marks]**

Q42. What is the difference between AWS CloudFormation and AWS Elastic Beanstalk? **[4 marks]**

Q43. What happens when one of the resources in an Auto Scaling Group cannot be launched? **[3 marks]**

Q44. Explain health check mechanisms in Elastic Load Balancing. **[5 marks]**

Q45. How can you add an existing EC2 instance to a new Auto Scaling Group? **[4 marks]**

Section E: MCQ-Type Questions (1–2 Marks Each)

Q46. Amazon EC2 belongs to which cloud service model? A) SaaS B) PaaS C) IaaS ✓ D) DaaS **[1 mark]**

Q47. AWS Elastic Beanstalk belongs to which cloud service model? A) IaaS B) PaaS ✓ C) SaaS D) NaaS **[1 mark]**

Q48. Which AWS service is used for serverless computing? A) Amazon EC2 B) AWS Lambda ✓ C) Amazon RDS D) Amazon VPC **[1 mark]**

Q49. What is the maximum execution timeout for an AWS Lambda function? A) 5 minutes B) 10 minutes C) 15 minutes ✓ D) 30 minutes **[1 mark]**

Q50. Which type of load balancer operates at Layer 7 (HTTP/HTTPS)? A) Network Load Balancer B) Gateway Load Balancer C) Application Load Balancer ✓ D) Classic Load Balancer **[1 mark]**

Q51. In Auto Scaling, which capacity cannot be exceeded? A) Desired B) Minimum C) Maximum ✓ D) Current **[1 mark]**

Q52. Which EC2 pricing option offers up to 90% discount but can be interrupted? A) On-Demand B) Reserved C) Spot ✓ D) Dedicated **[1 mark]**

Q53. AWS Lambda billing is based on: A) Per hour B) Number of requests and duration ✓ C) Per day D) Fixed monthly fee **[1 mark]**

Q54. Which storage type in EC2 loses data when the instance is stopped? A) EBS B) EFS C) Instance Store ✓ D) S3 **[1 mark]**

Q55. Elastic Beanstalk supports which of the following platforms? A) Python and Node.js B) Java and .NET C) Go and Docker D) All of the above ✓ **[1 mark]**

UNIT 9: QUICK REVISION SUMMARY

Key Definitions — One Liners

- IaaS: Cloud service providing virtualized hardware (compute, storage, network) over the internet.
- Amazon EC2: AWS IaaS service providing resizable virtual machines (instances) in the cloud.
- AMI: Amazon Machine Image — template containing OS + software for launching EC2 instances.
- EBS: Elastic Block Store — persistent block storage for EC2 instances.
- Security Group: Virtual stateful firewall controlling inbound/outbound EC2 traffic.
- ELB: Elastic Load Balancing — distributes traffic across multiple targets for availability.
- ALB: Application Load Balancer — Layer 7, path/host-based routing for web apps.
- NLB: Network Load Balancer — Layer 4, ultra-low latency, millions of requests/sec.
- Auto Scaling: Automatically adjusts EC2 capacity based on demand to optimize cost and availability.
- ASG: Auto Scaling Group — logical grouping of EC2 instances with min/max/desired capacity.
- Serverless Computing: Cloud model where cloud manages servers; user just writes and runs code.
- AWS Lambda: Serverless, event-driven compute service; runs code without managing servers.
- Lambda Trigger: Event source that invokes a Lambda function (S3, API Gateway, SQS, etc.)
- Elastic Beanstalk: PaaS — deploys and manages web apps automatically on AWS infrastructure.
- Web Server Tier: Beanstalk tier for handling HTTP requests using EC2 + ELB + Auto Scaling.
- Worker Tier: Beanstalk tier for processing background tasks from SQS queues.

Formula Summary — Lambda Pricing

- Cost = (Number of requests × $0.20 per million) + (GB-seconds × $0.0000166667)
- GB-seconds = Memory (in GB) × Duration (in seconds)
- Free tier: 1 million requests/month + 400,000 GB-seconds/month

Remember These Numbers

Parameter	Value
Lambda max timeout	15 minutes
Lambda memory range	128 MB – 10,240 MB
Lambda free requests/month	1 million

Lambda free GB-seconds/month	400,000
EC2 Spot savings vs On-Demand	Up to 90%
EC2 Reserved savings vs On-Demand	Up to 75%
Auto Scaling default cooldown	300 seconds
ELB Health Check default interval	30 seconds
EC2 default root EBS volume	8 GB
SSH default port	22
HTTP default port	80
HTTPS default port	443